TractionTherm

Django backend design · modules, models, tables, services and APIs

This document describes the server that will replace the browser-only storage of the TractionTherm motor-monitoring web UI. It is written for the backend team: it lists the Django apps to create, every model and database table with its fields, the services that hold the business rules, and the REST and WebSocket APIs the existing front end needs.

Everything here is derived from what the current front end does today (the pages in this folder and `assets/js/app.js`), so the backend can be built without changing how the app looks or behaves.

Item	Value
Product	TractionTherm · traction-motor temperature monitoring (heat-run tests)
Front end	Static HTML / CSS / JavaScript, 13 pages, no framework
Target backend	Python 3.12, Django 5.x, Django REST Framework, PostgreSQL 16
Real-time	Django Channels (WebSockets) with Redis
Background work	Celery with Redis (evaluation, archiving, exports, device timeouts)
Status	Design for implementation · version 1.45

Contents

1. What the system does 4

1.1 Domain in one page . . . 4

1.2 Pages and what each needs from the server . . . 4

2. Architecture 6

2.1 Technology choices . . . 6

2.2 Project layout . . . 6

3. Modules (Django apps) 7

4. Models and database tables 7

4.1 accounts . . . 7

4.2 devices . . . 8

4.3 motors . . . 9

4.4 testing . . . 9

4.5 telemetry . . . 10

4.6 conditions . . . 11

4.7 activity . . . 11

4.8 Relationships at a glance . . . 12

5. Services (business logic) 12

5.1 Evaluation rules (port of the front-end logic) . . . 14

5.2 Test report sheet (PDF and Excel) . . . 14

5.3 Background jobs (Celery beat) . . . 15

6. REST API 16

6.1 Auth and current user . . . 16

6.2 Devices . . . 16

6.3 Motors . . . 17

6.4 Tests (runs) . . . 17

6.5 Results and exports . . . 18

6.6 Settings . . . 18

6.7 Users, roles, activity . . . 19

6.8 Device ingestion (machine to machine) . . . 19

7. Live updates (WebSockets) 20

8. Permissions 20

9. Moving the front end to the API 21

10. Security and operations 21

10.1 Runtime services (what runs on the server) 22
10.2 Settings each service needs (environment) 22
11. Build plan **22**
Acceptance tests to port 23

1. What the system does

Traction motors are heat-run tested in a loco shed. A test device sits at the bench and reads temperatures; the web app shows them live, judges each motor's condition and keeps the results.

1.1 Domain in one page

- **Device**: a Wi-Fi data logger with **2 channels**. It can test at most 2 motors at once and has **5 terminals**: 2 per channel (T1, T2) and 1 shared ambient sensor (TA).
- **Motor**: identified by Motor ID and a unique **Stator No.** (plus model and locomotive). A motor can be tested many times over its life.
- **Motor type**: **Three-phase** (stored as AC; sensors DE = drive end, NDE = non-drive end) or **Conventional** (stored as DC; PE = pinion end, CE = commutator end). The UI shows only the names Three-phase motor / Conventional motor. The type decides the sensor names on every page and report, for both types the same limits apply. Asked when a motor is registered, and when a channel is switched on without a motor.
- **Test** (a run): one motor on one device channel, from Switch on until Stop. Every test has its own **Test ID** (e.g. TEST-0007). The stator stays the same; the Test ID changes each run.
- **Reading**: one sample of T1, T2 and TA every **real-time interval** (Settings, default 2 s, 1-60 s). TA comes from the device's one ambient sensor, so both motors on a device have the same TA at the same moment. A per-minute trend is kept for the whole test; results and reports list one reading per **interval time** (Settings, default 15 min).
- **Vibration readings**: SKF readings typed in when a motor is stopped: velocity (mm/s) and acceleration envelope (gE) on each side (DE / NDE or PE / CE); saved with the test and printed on its report sheet.
- **Condition**: **Unstable** as soon as any check reaches its Unstable limit; **Stable** only when the temperatures have **settled** (the last 5 recorded readings each changed by less than 0.5 °C on both sensors) and every check is below its Processing limit; **Processing** otherwise (still heating, or between the limits). One set of limits for both motor types; the settle rule is set in Settings (default 5 readings, 0.5 °C).
- **Switch on without a motor**: an operator can start reading a free channel straight away; the run gets a temporary ID (NEW-001) and the real motor is attached later.

1.2 Pages and what each needs from the server

Page	Purpose	Server data used
Sign in (index)	Username / e-mail and password	auth
Dashboard	Counts by condition, alarms, motor temperature chart (T1 / T2 per motor against the temperature limit, predicted final temperature) with a table that also shows test time of the maximum hours	live summary, conditions
Monitor	A live widget per device: its two channels with TA, the device's one ambient sensor, shown once in the middle as common to both; recent motor activity per device. Under each motor sensor (here and on the Motor, Device and Dashboard pages): its rise above ambient and its change between the last two recorded readings (interval time apart, the report sheet's DIFF; each sensor box is split by a diagonal from bottom-left to top-right: temperature, label and rise in the top-left half, 'Rate of change per 15 min' with its value (e.g. +0.2 °C, same size as the temperature) in the bottom-right half; green once below settle_diff; until the 2nd recorded reading (interval time after the test start) it shows '—' with '2nd reading in N min')	devices, live readings (incl. last two recorded readings), activity

Page	Purpose	Server data used
Device	3D view of one device, two channels side by side with the common TA in the middle (real-time graph with its table, range and interval picker shared with Results and the Motor page), test history, activity	device, runs, readings, activity
Motor	One motor: live 3D, readings, real-time chart (range and interval picker, shared with Results and the Device page) and comparison, test history; vibration readings asked when stopped; a message when a running test is past its maximum test hours (also on the Monitor widget and the Device page)	motor, run, readings, evaluation
Results	Completed tests only (stopped or archived; each Stable or Unstable) · tests in progress are followed on the Monitor, Device and Motor pages, and asking for one here shows 'TEST-… is still in progress' with links to it. Test results table with the key columns only, the result first: Test ID, motor, condition, started, duration, rise, device · channel, and model / type on wide screens (no state column: every row is completed). Below 1280 px wide the rows become cards (two per row on tablets), 10 a page, condition filters, custom and quick rules, readings log, CSV (every column) / print; clicking a test opens its own view: head (Test ID, Edit test, Download PDF / Excel, Close), full details (every column), a whole-test graph (every stored reading from the start, live every sample while the test runs) and, for the motor's current test, a real-time graph with its table, latest values and °C/min, where the user picks the range (last 5 / 15 / 30 min, last 1 h, whole test) and the interval (live at the real-time interval, or every 10 s, 30 s, 1 min, 5 min when coarser than it; an interval needs at least 3 points in the range; it keeps updating live at any interval). The same picker is on the Motor page and the Device page real-time graphs and the choice is shared by all of them (remembered per browser) — both on screen only, never in the downloaded report, then its sheet and graph (on screen the sheet's readings table also has a TA °C column and its graph a TA line; the downloaded sheet is unchanged), a change log of the test, and the motor's result history (every test of that motor) with its own Download CSV; the filters are hidden in that view; there is no separate per-motor test-by-test list (the results table, filtered to a motor, is its history); PDF / Excel report sheets; before a report is downloaded, each stopped test in it without vibration readings asks for them (Save and continue, Skip, Skip all, Cancel; only for users with monitoring.update, never for a running test)	tests, readings, reports, test changes
Devices	Register and edit devices, see their motors	devices CRUD
Motors	Register, edit, assign, delete motors	motors CRUD, assignment
Settings	Only the test settings, side by side in one row of boxes with one Save: maximum test hours, interval time, last how many readings, change less than (°C), temperature must not exceed (°C), and ask for vibration on stop. Below them the Processing / Stable / Unstable rules with the current values and a live line per running motor. One set for three-phase and conventional motors	settings
Users	Accounts, enable / disable	users CRUD
Roles & permissions	Roles and their CRUD permissions	roles CRUD
Activity log	Who did what, filter by device / user / category, CSV	activity

Page	Purpose	Server data used
My profile	The signed-in user's own details (read-only) and a password change	auth/me, change password

2. Architecture

```
 Test device (2 channels, T1 T2 TA)                    Browser (existing pages)
      |  HTTPS POST /api/ingest/  (or MQTT)              |  REST /api/v1/...      WebSocket /ws/...
      v                                                  v
+-------------------------------- Django (ASGI: Daphne / Uvicorn) ---------------------------------+
|  telemetry.ingest  ->  readings table   ->  conditions.evaluate  ->  channels group "device.<id>"  |
|  DRF viewsets (devices, motors, tests, settings, users, roles, activity, reports)                 |
+---------------------------------------------------------------------------------------------------+
      |                     |                          |
 PostgreSQL 16         Redis (channel layer,      Celery workers
 (+ TimescaleDB         Celery broker, cache)      (minute trend, archiving, offline
  for readings)                                     detection, exports)
```

2.1 Technology choices

Concern	Choice	Why
Web framework	Django 5 + Django REST Framework	Admin, ORM, migrations, auth and permissions out of the box
Database	PostgreSQL 16; TimescaleDB extension for readings	A 2 s sample per channel is about 43,000 rows a day per device; hypertables keep it fast
Live updates	Django Channels + channels-redis	The UI refreshes at the real-time interval (Settings, default 2 s); push instead of polling
Background jobs	Celery + Redis, celery-beat	Minute trend, device-offline detection, large exports
Auth	Django session auth (same-site) or SimpleJWT for devices	Browser: secure HttpOnly session cookie. Devices: per-device API keys
API docs	drf-spectacular (OpenAPI 3)	Generated schema for the front end and device firmware
Exports	csv module; ReportLab for the PDF report sheet; openpyxl for the Excel sheet with a native line chart	Report sheets match the front end's PDF / Excel downloads (section 5.2)

2.2 Project layout

```
tractiontherm/                # Django project
  settings/  base.py dev.py prod.py
  asgi.py  urls.py  celery.py
apps/
  accounts/      users, roles, permissions, sign-in
  devices/       devices, channels, device API keys
  motors/        motors (identity: Motor ID, Stator No.)
  testing/       test runs, Test ID sequence, switch on / stop / assign / channel / terminals
  telemetry/     readings, minute trend, ingestion, live push
  conditions/    test settings, evaluation, heating model
  reports/       results queries, CSV and PDF exports
  activity/      audit log
  core/          shared mixins, permissions, pagination, errors
```

3. Modules (Django apps)

App	Owns	Depends on
accounts	User, Role, RolePermission, sign-in / sign-out, permission checks	core
devices	Device, DeviceApiKey, online status, device switcher list	accounts
motors	Motor (identity and details)	accounts
testing	TestRun, TestIdSequence; Switch on, Stop, Assign, Change channel, Change terminals, Edit test	devices, motors, telemetry, conditions, activity
telemetry	Reading (2 s), TrendPoint (1 min), ingestion endpoint, live WebSocket push	devices, testing
conditions	TestSettings (one set for both motor types), evaluation (settle rule, maximum hours, temperature limit) and heating model	telemetry
reports	Results queries, CSV / PDF export	testing, telemetry, conditions
activity	ActivityEntry (append only) and its device tags	accounts
core	BaseModel (timestamps), permission classes, error format, pagination	-

4. Models and database tables

Field types are Django field types. Table names use the `app_model` default. Every table also has `created_at` and `updated_at` from `core.BaseModel` unless stated.

4.1 accounts

User

Table `accounts_user`

Field	Type	Rules / notes
id	BigAutoField	Primary key
username	CharField(150)	Unique, lower case; sign-in accepts username or e-mail
email	EmailField	Unique
name	CharField(120)	Display name, e.g. Arun Prakash
role	ForeignKey(Role, PROTECT)	Exactly one role per user
password	CharField	Django hash (PBKDF2 / Argon2). Never stored or sent in plain text
is_active	BooleanField	Disabled accounts cannot sign in; open sessions are ended
last_login	DateTimeField(null)	Set at sign-in

- Custom user model (`AUTH_USER_MODEL`), set before the first migration.

Role

Table `accounts_role`

Field	Type	Rules / notes
id	BigAutoField	Primary key
name	CharField(60)	Unique. Built in: Administrator, Supervisor, Operator, Viewer
description	CharField(200)	
is_builtin	BooleanField	Built-in roles cannot be deleted or renamed
is_admin	BooleanField	Administrator: always every permission, cannot be edited

- Custom roles are created by an administrator; a role in use by a user cannot be deleted.

RolePermission

Table `accounts_rolepermission`

Field	Type	Rules / notes
role	ForeignKey(Role, CASCADE)	
key	CharField(40)	One of the permission keys in section 8, e.g. motors.update

- Unique together (role, key). Granting create / update / delete also grants the area's view (the UI already enforces this).

4.2 devices

Device

Table `devices_device`

Field	Type	Rules / notes
id	CharField(20), primary key	Device ID, e.g. TMU-01 (printed on the unit)
name	CharField(60)	Unique, e.g. Device A
location	CharField(120)	e.g. Loco Shed · Bay 3
ip_address	GenericIPAddressField(null)	Last reported
firmware	CharField(20)	e.g. 2.4.1
channels	PositiveSmallIntegerField	Default 2 (CH 1, CH 2)
signal	PositiveSmallIntegerField(null)	Wi-Fi bars 0-4, last reported
last_seen	DateTimeField(null)	Updated by every ingest and heartbeat; online = seen within max(10 s, 3 × live_seconds), so a slow real-time interval doesn't make devices flicker offline between readings

- Online / offline is derived from last_seen (a Celery beat task marks devices offline and pushes the change).

DeviceApiKey

Table `devices_deviceapikey`

Field	Type	Rules / notes
device	ForeignKey(Device, CASCADE)	
key_hash	CharField(128)	Hash of the key the device sends in its Authorization header
label	CharField(60)	
revoked_at	DateTimeField(null)	

4.3 motors

Motor

Table `motors_motor`

Field	Type	Rules / notes
id	BigAutoField	Primary key (internal)
motor_id	CharField(40)	Unique, upper case, e.g. TM-4471. Can be renamed; history follows the row
stator_no	CharField(40, null)	Unique when set, upper case, e.g. ST-60681. Required except for a temporary motor
model	CharField(120)	e.g. 6FRA 6068 · 3-phase AC
locomotive	CharField(60)	e.g. WAG-9 #31245
motor_type	CharField(2)	AC (shown as Three-phase motor) or DC (shown as Conventional motor). Required; also chosen at Switch on for a temporary motor. Decides sensor names (DE / NDE or PE / CE) and the limits used
is_temporary	BooleanField	True for NEW-001 style motors created by Switch on before details are known

- Where a motor is now is not stored here: it is the device of its open TestRun, if any.
- Deleting a motor is refused while it has an open test; finished tests keep the motor (soft delete with `deleted_at` recommended).

4.4 testing

TestRun

Table `testing_testrun`

Field	Type	Rules / notes
id	BigAutoField	Primary key
test_id	CharField(24)	Unique, upper case, 3-24 letters / digits / dashes, e.g. TEST-0007
motor	ForeignKey(Motor, PROTECT)	
device	ForeignKey(Device, PROTECT)	Device the test ran on (editable later)
channel	PositiveSmallIntegerField(null)	1 or 2
state	CharField(10)	running / stopped / archived
started_at	DateTimeField	

Field	Type	Rules / notes
stopped_at	DateTimeField(null)	Set by Stop
motor_type	CharField(2)	Copied from the motor when the test starts, so an old test keeps its sensor names and limits if the motor type changes later
t1_measures	CharField(40)	What T1 measures in this test (default from the motor type), e.g. Drive end (DE) or Pinion end (PE)
t2_measures	CharField(40)	What T2 measures in this test
result	CharField(8, null)	ok / warn / crit at the end of the test, shown as Stable / Processing / Unstable
max_t1, max_t2, max_rise	DecimalField(5,1)	Peaks, kept for the summary and fast lists
started_by, stopped_by	ForeignKey(User, SET_NULL, null)	
vib_s1_mms, vib_s1_ge	DecimalField(6,3, null)	Vibration on sensor 1's side (DE or PE): velocity 0-100 mm/s and acceleration envelope 0-50 gE. Both set or both empty
vib_s2_mms, vib_s2_ge	DecimalField(6,3, null)	The same on sensor 2's side (NDE or CE)
vib_by, vib_at	ForeignKey(User, SET_NULL, null), DateTimeField(null)	Who entered them and when; only while the test is stopped

- **One running test per channel**: partial unique index on (device, channel) WHERE state = 'running'.
- **One open test per motor**: partial unique index on (motor) WHERE state IN ('running', 'stopped').
- A stopped test is the motor's 'current' test until a new test starts; then it becomes archived.
- A Test ID once used (current or archived) can never be used again.

TestIdSequence

Table `testing_testidsequence`

Field	Type	Rules / notes
id	SmallAutoField	Single row
last_number	PositiveIntegerField	Highest TEST-NNNN number used; next free is last_number + 1, zero padded to 4

- Read and increment inside `select_for_update()`. Custom Test IDs typed by users don't move the sequence unless they follow TEST-NNNN.

4.5 telemetry

Reading

Table `telemetry_reading`

Field	Type	Rules / notes
test_run	ForeignKey(TestRun, CASCADE)	Indexed together with time
time	DateTimeField	Device time, UTC; TimescaleDB partition column
t1, t2, ta	DecimalField(5,1)	°C, one decimal (PT100-class sensors). ta is the device's one ambient sensor: the same value for both channels in a sample

- Written at the real-time interval per running channel; index (test_run, time). Raw readings kept 30 days (retention policy).

TrendPoint

Table `telemetry_trendpoint`

Field	Type	Rules / notes
test_run	ForeignKey(TestRun, CASCADE)	
minute	DateTimeField	Start of the minute; unique together (test_run, minute)
t1, t2, ta	DecimalField(5,1)	Reading at that minute (what the charts and downloads use)

- Kept for the life of the test (the front end keeps at most 600 points per run; the server has no such limit).

4.6 conditions

TestSettings

Table `conditions_testsettings`

Field	Type	Rules / notes
max_test_hours	DecimalField(4,1)	Default 4, 0.5-48 in half hours: Stable must be reached within this time; a test may run on past it (the pages then show a message)
interval_minutes	PositiveSmallIntegerField	Default 15, 1-240 ('Interval time'): one reading per this many minutes from the test start, in results, reports and the settle rule
settle_readings	PositiveSmallIntegerField	Default 5, 2-20 ('Last how many readings'): how many recorded readings in a row must have settled
settle_diff	DecimalField(4,1)	Default 0.5 °C, 0.1-10 ('Change less than'): each of them differs from the one before by less than this, on both motor sensors
max_temp	DecimalField(5,1)	Default 100 °C, 20-250 ('Temperature must not exceed'): the hotter of T1 / T2 above it makes the test Unstable at once
live_seconds	PositiveSmallIntegerField	Default 2, 1-60 ('Real-time interval'): how often devices send readings (returned in the heartbeat reply); live values, real-time graphs and alarms update at this rate. Open pages reload when it changes (their timers restart)
ask_vibration_on_stop	BooleanField	Default True: ask for the vibration readings when a motor is stopped
updated_by	ForeignKey(User, SET_NULL, null)	

- Single row, set on the Settings page with one Save (seven boxes side by side: the six values above and the vibration question). Rejected (400) when settle_readings × interval_minutes > max_test_hours × 60: no test could become Stable. Replaces ConditionLimits (Processing / Unstable pairs) and TerminalLimit (min / max per sensor), which are gone.

Sensor names are not stored: they come from the motor type (three-phase / AC: DE / NDE, conventional / DC: PE / CE). Lists that mix both types use the slot names T1 / T2.

4.7 activity

ActivityEntry

Table `activity_activityentry`

Field	Type	Rules / notes
id	BigAutoField	
time	DateTimeField	Server time, indexed
category	CharField(10)	auth, device, motor, test, limits, user, role, report, access
text	TextField	Human sentence, e.g. 'Stopped monitoring TM-4471 on Device A CH 1 · test TEST-0001'
user	ForeignKey(User, SET_NULL, null)	Plus copies of user name, username and role at the time
devices	ManyToManyField(Device)	Devices the event belongs to (both devices for a move). Powers the per-device lists
motor, test_run	ForeignKey(null, SET_NULL)	Optional links for 'open the motor' and filtering. test_run is set on every entry about a test (switch on / off, edit, channel, terminals, vibration, report download) and feeds the test's change log

- Append only: no update or delete through the API or admin. No updated_at.

4.8 Relationships at a glance

```
Role 1 --- * User                Device 1 --- * TestRun * --- 1 Motor
Role 1 --- * RolePermission      TestRun 1 --- * Reading
                                 TestRun 1 --- * TrendPoint
ActivityEntry * --- * Device     ActivityEntry * --- 0..1 Motor / TestRun / User
TestSettings (1 row)
TestIdSequence (1 row)
```

5. Services (business logic)

Views stay thin: they validate input with serializers and call a service. Every service that changes data runs in `transaction.atomic()`, checks the permission, writes an ActivityEntry and publishes a WebSocket event.

Service	Function	Rules it enforces
TestIdService	next_id(), reserve(test_id)	Format and uniqueness against all current and archived tests; sequence under row lock
MonitoringService	switch_on(device)	Device online; a free channel; creates a temporary motor NEW-NNN and a running test with the next Test ID
	start(motor, device, test_id)	Free channel on that device; archives the motor's previous stopped test; copies sensor 'measures' defaults into the test
	stop(test_run)	Sets stopped_at, computes result and peaks; channel becomes free; the UI then asks for the vibration readings
VibrationService	record(test_run, side1, side2)	Only on a stopped test; each side both values or none, at least one side; velocity 0-100 mm/s, envelope 0-50 gE, 3 decimals; activity entry with the values
AssignmentService	assign(motor, device, test_id)	If the motor runs elsewhere: stop there first (UI asks to confirm); then start on the target
	claim(temp_test, motor, test_id)	'Add details' on a switched-on run: the chosen motor takes over the run and readings; the temporary motor is removed
ChannelService	change(test_run, channel)	Same device. Other channel free: move. Busy: swap the two running tests. Readings stay with each test

Service	Function	Rules it enforces
TerminalService	change(test_run, t1_measures, t2_measures, swap)	swap=true exchanges t1 / t2 in all Reading and TrendPoint rows of that test and its peaks
TestEditService	edit(test_run, ...)	Edit test window: terminals (what T1 / T2 measure, from a dropdown) always; swap readings only while the test runs; vibration on a stopped test; motor details (which apply to all the motor's tests) always. The test itself (Test ID, device, channel) is shown but not editable. The channel of a running test is changed with Change channel
IngestService	ingest(device, samples)	Authenticates the device key; maps channel to its running test; stores readings; updates last_seen; rolls the minute trend; triggers evaluation and push. Heartbeat replies carry live_seconds
EvaluationService	evaluate(test_run)	Checks with the limits of the test's motor type, the settle rule and the condition (section 5.1); heating model; returns the same structure the UI shows
ReportService	results(filters), readings(filters), sheet(test_run), report_pdf(tests), report_xlsx(tests), export_csv()	One row per test with its calculated results; readings at the recording interval from Settings (counted from each test start; the readings log may ask for another interval); rules (all / any); the report sheet of section 5.2 as PDF or Excel
ActivityService	log(category, text, user, devices, motor, test)	Tags devices from the event; immutable
AuthService	sign_in(), sign_out(), change_password(user, current, new)	Username or e-mail; disabled accounts refused; failed attempts logged and rate limited. Password change checks the current password, runs Django's password validators (at least 6 characters, different from the current one), keeps this session and ends the user's other sessions; success and failure are logged (category auth)
DeviceService	create(data), update(device, data), delete(device)	ID 3-16 letters / numbers / dashes, unique; name unique; IP four numbers 0-255, unique; each failure a clear message (400). delete: refused (409) while a test runs on it, naming the motors to stop; otherwise its motors become unassigned and their completed tests are kept. Activity entry with what changed
MotorService	register(data, device=None, test_id=None), update(motor, data), delete(motor)	Motor ID and stator No. unique (message names the motor that has it); type AC / DC (shown three-phase / conventional); register on a device starts a test (MonitoringService.start); a rename keeps all its tests; delete refused while it runs
SettingsService	get(), update(values, user)	All values checked together (ranges in section 4.6; settle_readings × interval_minutes must fit in max_test_hours); one activity entry listing every change; broadcasts settings.changed so open pages apply it at once; a new live_seconds reaches devices in their next heartbeat reply
ResultsService	completed(filters), readings(test_run, range, every)	Results list completed tests only (stopped or archived; result Stable or Unstable, a result saved as Processing counts as Unstable). readings(): the raw samples of the last hour plus the minute trend before them, one reading per interval counted back from the newest (the real-time and whole-test graphs)
ChangeLogService	changes(test_run)	The ActivityEntry rows linked to the test run, newest first (earlier Test IDs included): the test's change log on Results
UserService	create(data), update(user, data), set_active(user, on), delete(user)	Name, username, e-mail required; username and e-mail unique; e-mail valid; password at least 6 characters (Django validators); no one can delete or disable themselves; the last active administrator can't be removed or demoted; disabling ends the user's sessions

Service	Function	Rules it enforces
RoleService	create(name, desc, from_role), update(role, ...), set_permissions(role, keys), delete(role)	Name 2-30 characters, unique; built-in roles (Administrator, Supervisor, Operator, Viewer) are never deleted or renamed, Administrator's permissions are fixed; a role with users can't be deleted; create / update / delete imply view; one activity entry with the permissions added and removed

5.1 Evaluation rules (port of the front-end logic)

Check	Value	Effect
Motor temperature	hot = max(T1, T2); peak = highest hot in the test so far	peak > max_temp: Unstable (at once, also while running)
Temperatures settled	the last settle_readings recorded readings (interval apart), counting only readings up to start + max_test_hours: each |reading − previous reading| on T1 and T2	All < settle_diff: settled = Stable. Otherwise 'Not settled yet' = Processing
Test time	running time against max_test_hours	max_test_hours reached, or the test stopped, without settling: Unstable ('Not settled in time')

A test's condition: **Unstable** when the temperature went above the limit, or the maximum hours passed without settling; otherwise **Stable** once the temperatures settled within the maximum hours; otherwise **Processing** (still running, not settled yet). A test may keep running past its maximum hours: its condition no longer changes from settling, and the pages say 'Maximum test hours exceeded (N hours) · result … · stop the test when ready'. A stopped (completed) test is **Stable** if it settled within the maximum hours, otherwise **Unstable** ('Stopped before settling'): a result is never Processing. Results saved before this rule as Processing are shown as Unstable. A device offline shows 'Offline' on the live pages. Wherever a motor's state is shown (Motors list, Monitor, Device and Motor pages), a stopped motor shows that final result with a small 'Stopped' tag (monitoring switched off) rather than 'Stopped' alone; it links to that completed test on Results, where it is listed as soon as the motor is stopped. The rise above ambient, T1 / T2 difference, comparison with the other motor, predicted final rise, terminal min / max and probe checks are no longer part of the condition (rise, difference and the heating model's predicted final temperature are still shown as information).

Condition types (Results filter and rule): **Temperature above the limit**, **Not settled in time** (test summary only). The Results condition filter, rules and summary use Stable / Unstable only. In the readings log one reading is Unstable when its hotter motor sensor is above max_temp, otherwise Stable.

Heating model

For each of T1 and T2 the rise above ambient over test time t (minutes) is fitted to **rise(t) = A − (A − r0)·e$^{-t/\tau}$** by least squares (grid search on τ from 1 to 600 min, closed form for A and r0). The fit is **confident** when $R^2 \geq 0.8$ and the test has run at least 0.4·τ. From it: predicted final temperature = TA + A; time to equilibrium t = τ·ln(60·(A − r0) / (2·τ)); time until the hotter sensor would pass max_temp. The heating model also estimates when the motor will settle; the Stable decision itself uses the measured settle rule above.

> Run evaluation in a Celery task after each minute trend point and cache the latest result per test in Redis; the live push at the real-time interval re-uses the cached model and only recomputes the simple checks.

5.2 Test report sheet (PDF and Excel)

Every test has one report sheet with the fields of the shed's heat-run sheet, drawn in TractionTherm's own style (the app's table look and graph colours), not the reference sheet's look. The front end already makes it as PDF and Excel; the server version must give the same numbers. Rows are the readings at the recording interval from the test start (T + 0, T + 15, … min); both sensors use the names of the test's motor type.

Part	Contents / formula
Header	TM NO (motor ID), DATE (test start, DD-MM-YYYY)
Table	TIME (H:MM) · sensor 1 · DIFF · sensor 2 · DIFF. DIFF = reading − previous reading (0 on the first row); shown green when |DIFF| < settle_diff
Runtest duration	last row time − first row time (H:MM)
Initial / final ambient temp	TA of the first / last row
Final temp	each sensor on the last row
Temp raise	final temp − final ambient temp, per sensor
Max diff in last 1 hr temp readings	highest − lowest reading in the last 60 min of rows, per sensor
SKF reading (each side)	one row per side (DE / NDE or PE / CE): velocity mm/s under the first column, envelope gE under the second; only when vibration readings were entered
Graph	both sensors over the rows; title = TM NO; x axis 'Time (At 15min)'; y axis 'Temperature'
Nothing else on the sheet	no report title, generated-by line, result, test ID, type, device or notes: the sheet itself matches the shed's reference sheets
Full details (first, before the sheet)	every field of the test as on the Results page's Full details, in the same order: test ID, motor, stator, model / type, locomotive, device, channel, location, started, monitoring for, state, last T1 / T2 / TA, rise, T1 / T2 difference, predicted final, time constant, settled, the calculated results above, vibration per side, condition, reason. Sensor names are the test's own (DE / NDE or PE / CE); Δ and τ are spelled out (difference, time constant (tau)). PDF: a page of its own with a FULL DETAILS · test ID header band and label / value cells, 4 a row, the reason wrapped in the last cell, plus a 'Generated … · readings every N min' line. Excel: a FULL DETAILS band row at the top of the test's block, then one field a row, then the sheet with its chart (label A:E, value G:I, numbers as numbers)

Several tests (all tests of a motor, all motors) make one file: for each test its Full details page followed by its sheet page(s) in the PDF, the sheets one below another in a single Excel worksheet, each with its own chart. The Results page shows the same calculated results for each test (table columns, and the opened test laid out as its sheet).

5.3 Background jobs (Celery beat)

Scheduled work runs in the Celery worker; each job is idempotent (safe to run twice) and records failures in the logs and the monitoring alerts.

Job	When	What it does
roll_minute_trend	every minute	For every running test: writes the minute TrendPoint from the raw readings of that minute
evaluate_running_tests	after each trend point (every minute)	Runs EvaluationService for every running test, caches the result in Redis and pushes it; the live push in between re-uses the cache
watch_max_hours	every minute	A running test that passes max_test_hours gets its final condition (Stable if settled in time, else Unstable 'Not settled in time'), an activity entry and a test.changed push with the 'maximum test hours exceeded' notice; it keeps running until stopped
detect_offline_devices	every 10 s	Marks a device offline when not seen within max(10 s, 3 × live_seconds); pushes device.status; alerts if a test was running on it
purge_raw_readings	nightly 02:00	Deletes raw Reading rows older than 30 days (the minute trend, tests and activity are kept)
build_export	on request	Large PDF / Excel / CSV exports (all motors, many tests) are built here and offered as a download link when ready

Job	When	What it does
yearly_archive	1 January	Moves the previous year's tests, trend and activity to archive tables (still readable on Results)

6. REST API

Base path `/api/v1/`. JSON, UTC ISO-8601 times, all values in °C with one decimal: temperatures, rises above ambient, differences and rates (°C/min) alike, never kelvin. Errors use `{"error": {"code": "...", "message": "...", "field": "..."}}` with 400 (validation), 401 (not signed in), 403 (no permission), 404, 409 (conflict such as a busy channel or a used Test ID).

6.1 Auth and current user

Method	Path	Permission	Does
POST	/auth/sign-in/	public	Body: login (username or e-mail), password. Sets the session cookie; returns the user with role and permission keys
POST	/auth/sign-out/	signed in	Ends the session
GET	/auth/me/	signed in	Current user, role, permission keys (the UI hides buttons with these). Also feeds My profile: name, username, e-mail, role, status, last sign-in
POST	/auth/me/password/	signed in	Body: current_password, new_password. 400 with a message if the current password is wrong or the new one fails validation; 204 on success. Rate limited like sign-in

6.2 Devices

Method	Path	Permission	Does
GET	/devices/	devices.view or monitor.view	List with online state, channels in use, running motors
POST	/devices/	devices.create	Create (id, name, location, ip, firmware). 400 with a message: missing field; id not 3-16 letters / numbers / dashes; id, name or IP already used by another device; IP not four numbers 0-255
GET	/devices/{id}/	devices.view	One device with both channels and its stopped motors
PATCH	/devices/{id}/	devices.update	Edit name, location, IP (same checks as create)
DELETE	/devices/{id}/	devices.delete	409 while a test is running on it (the message names the motors to stop). Otherwise its motors become unassigned and their completed tests are kept
POST	/devices/{id}/switch-on/	monitoring.create	Body: motor_type (AC = three-phase / DC = conventional, required). Start reading the free channel without a motor (temporary NEW-NNN of that type)
GET	/devices/{id}/activity/?limit=5	activity.view	Recent motor and test events of this device

6.3 Motors

Method	Path	Permission	Does
GET	/motors/?device=&search=&condition=	motors.view	List with where it is, its current test and live condition
POST	/motors/	motors.create	Register (motor_id, stator_no, motor_type, model, locomotive); optional device + test_id to start straight away
GET	/motors/{id}/	monitor.view	Details, current test, live readings and evaluation
PATCH	/motors/{id}/	motors.update	Edit details (applies to all its tests)
DELETE	/motors/{id}/	motors.delete	Refused while it has an open test
POST	/motors/{id}/assign/	motors.update	Body: device, test_id. Starts a test on the device's free channel (moves it if running elsewhere)
GET	/motors/{id}/tests/	monitor.view	Its tests, newest first (Test history)

6.4 Tests (runs)

Method	Path	Permission	Does
GET	/tests/?motor=&device=&condition=&search=	reports.view	Results table: one row per completed test (a stopped current test and the earlier ones; running tests are left out). condition is ok / crit only
GET	/tests/{test_id}/	reports.view	One test with summary, peaks, terminals, result
PATCH	/tests/{test_id}/	motors.update	Edit test: t1/t2 measures (same dropdown values and rules as /terminals/; when motor_type changes in the same request they are checked against the new type; allowed on running and finished tests) and swap (running test only: 409 once the result is generated), motor details (stator, model, locomotive, type), and vibration {side1, side2} on a stopped test (needs monitoring.update as well; 409 while running; null clears a side). Every changed field goes into the activity entry. test_id, device and channel are read-only: 400 if sent
POST	/tests/{test_id}/start/	monitoring.update	Switch a stopped motor on again (creates a new test with the next Test ID)
POST	/tests/{test_id}/stop/	monitoring.update	Stop. Reply includes ask_vibration
PUT	/tests/{test_id}/vibration/	monitoring.update	Body: side1 {mms, ge} or null, side2 {mms, ge} or null. 409 while the test is running
POST	/tests/{test_id}/claim/	motors.update	Add details on a switched-on run: body motor (existing) or new motor fields, test_id
POST	/tests/{test_id}/channel/	monitoring.update	Body: channel. Moves or swaps

Method	Path	Permission	Does
POST	/tests/{test_id}/terminals/	monitoring.update	Body: t1_measures, t2_measures, swap. Each measures value is picked from a dropdown: one of the motor type's measuring points (three-phase: Drive end (DE) / Non-drive end (NDE); conventional: Pinion end (PE) / Commutator end (CE)) or the value already saved; 400 otherwise, and 400 if both are the same. Allowed on a running or stopped test; swap only while running (409 once the result is generated)
GET	/tests/{test_id}/readings/?from=&to=&every=	reports.view	Trend points, or the raw 2 s samples with every in seconds (every=2s, 10s, 30s, 60s, 300s: one reading per interval counted back from the newest; raw samples are kept at least the last hour, older parts of a range come from the minute trend). every=15 (minutes) returns one reading per 15 min from the test start, as the report sheet lists them
GET	/tests/{test_id}/evaluation/	monitor.view	Checks, condition, heating model
GET	/tests/{test_id}/report.pdf · report.xlsx	reports.create	This test's report sheet (section 5.2)
GET	/tests/{test_id}/changes/	reports.view	Change log of one test: the ActivityEntry rows linked to its TestRun (so earlier test IDs are included), newest first: time, user name and username, text
GET	/test-ids/next/	monitoring.create	Next free Test ID to pre-fill forms

6.5 Results and exports

Method	Path	Permission	Does
GET	/reports/readings/?range=60&every=15&...	reports.view	Readings log (Temperature readings view). every defaults to the recording interval
POST	/reports/export/	reports.create	Body: type (summary / readings), filters, rules; returns CSV (or PDF for print)
GET	/motors/{id}/report.pdf · report.xlsx	reports.create	Every test of one motor: one sheet each, in one file. Not used by the Results page any more (its per-motor Download all tests was removed); keep for API clients or drop
GET	/reports/all-motors.pdf · .xlsx	reports.create	The current test of every motor, one sheet each

6.6 Settings

Method	Path	Permission	Does
GET	/settings/	limits.view	The test settings (one set for both motor types)

Method	Path	Permission	Does
PUT	/settings/	limits.update	Body: max_test_hours, interval_minutes, settle_readings, settle_diff, max_temp, live_seconds, ask_vibration_on_stop (all, validated together; 400 if the settle rule can't fit in the maximum hours). One activity entry listing what changed. Broadcasts settings.changed on /ws/live/ so every open page applies the new values at once (no reload): conditions and reasons, report sheets at the new interval, the Results 'Reading every' default, limit lines and the Dashboard chart

6.7 Users, roles, activity

Method	Path	Permission	Does
GET / POST	/users/	users.view / users.create	List, create
PATCH / DELETE	/users/{id}/	users.update / users.delete	Edit, enable / disable, delete (not yourself, not the last administrator)
GET / POST	/roles/	roles.view / roles.create	List with permission keys, create custom role
PATCH / DELETE	/roles/{id}/	roles.update / roles.delete	Built-in roles (Administrator, Supervisor, Operator, Viewer) can't be deleted or renamed; Supervisor / Operator / Viewer keep editable descriptions and permissions; Administrator can't be edited. A custom role in use by users can't be deleted
GET	/permissions/	roles.view	The permission catalogue (area, action, label)
GET	/activity/?category=&user=&device=&period=&search=	activity.view	Paged, newest first
GET	/activity/export.csv	activity.view	Same filters

6.8 Device ingestion (machine to machine)

Method	Path	Permission	Does
POST	/ingest/	device API key	Body below. 204 on success; 409 if the channel has no running test (samples are dropped and reported)
POST	/ingest/heartbeat/	device API key	ip, firmware, signal; keeps the device online when idle.

```
POST /api/v1/ingest/                Authorization: Device <api-key>
{
  "device": "TMU-01",
  "samples": [
    {"time": "2026-11-02T06:31:04Z", "channel": 1, "t1": 50.2, "t2": 53.1, "ta": 31.0},
    {"time": "2026-11-02T06:31:04Z", "channel": 2, "t1": 57.0, "t2": 59.0, "ta": 31.0}
  ]
}
```

Devices may batch several seconds of samples. The same TA reading is sent with both channels (one ambient sensor per device).

7. Live updates (WebSockets)

Socket	Joined by	Messages pushed
/ws/live/	Monitor, Dashboard	device.status (online, signal), reading (device, channel, t1, t2, ta, condition) at the real-time interval, test.changed, settings.changed (the new test settings)
/ws/devices/{id}/	Device page	readings for both channels, test.changed, activity.new
/ws/tests/{test_id}/	Motor page, Results (opened test: whole-test and real-time graphs)	reading, evaluation (every minute), test.changed
/ws/activity/	Monitor (per-device lists)	activity.new with its device ids

Sockets use the same session as the REST API and check `monitor.view` (and `activity.view` for activity). Groups: `device.<id>`, `test.<test_id>`, `activity`.

8. Permissions

Permissions are keys `area.action`. The server checks them on every request (a DRF permission class reads the key from the view); the front end only hides buttons. Administrator always has all keys.

Key	Allows	Admin	Super.	Oper.	Viewer
monitor.view	Live readings, dashboard, motor pages	Yes	Yes	Yes	Yes
monitoring.create	Switch on a free channel	Yes	Yes	Yes	
monitoring.update	Start / stop, change channel, change terminals	Yes	Yes	Yes	
motors.view / create / update / delete	Motors page, register, edit (incl. Edit test, assign), delete	Yes	Yes	view	view
devices.view / create / update / delete	Devices and device pages, add, edit, delete	Yes	Yes	view	view
limits.view / update	Settings page, change the test settings and the vibration question	Yes	Yes	view	view
reports.view / create	Results page, download and print	Yes	Yes	Yes	view
activity.view	Activity log and recent-activity lists	Yes	Yes		
users.view / create / update / delete	Users	Yes			
roles.view / create / update / delete	Roles & permissions	Yes			

Defaults match the current front end; administrators can change any role except Administrator and create custom roles. My profile needs no permission key: every signed-in user can see their own details and change their own password, but name, username, e-mail and role are changed only through /users/ (users.update).

9. Moving the front end to the API

Today every page keeps its data in the browser's localStorage. Each key maps to an endpoint; replace the `store.get / store.set` calls in `app.js` with fetch calls and the simulated `tick()` with the WebSocket.

Browser key today	Holds	Replace with
tt-devices-v6	Devices, their motors, current tests, test history	/devices/, /motors/, /tests/
tt-spare-v1	Motors not on a device	/motors/?unassigned=1
tt-test-seq	Last Test ID number	/test-ids/next/
tt-limits-v5, tt-recording-v1, tt-vibration-v1	Test settings: maximum hours, settle rule, temperature limit; interval time; ask for vibration on stop	/settings/
tt-users-v2, tt-roles-v1, tt-grants-v3	Users (with plain passwords), roles, permissions	/users/, /roles/, /auth/me/
tt-activity-v1	Activity log (max 2000)	/activity/
tt-session (sessionStorage)	Signed-in user id	Server session cookie + /auth/me/
tick() at the live interval	Simulated readings	/ws/live/ and /ws/tests/{id}/

Graph and table. The Dashboard rise chart, the real-time graphs and the motor comparison show a table of the same numbers below the graph. No extra endpoints are needed: the tables use the same data as the graphs.

Pagination. Every list table (results, readings log, motors, devices, users, activity, test history, the tables under the graphs) shows 10 rows a page with Prev / page numbers / Next. List endpoints therefore take **page** and **page_size** (default 10, max 200) and return **count**, **next**, **previous** and **results** (DRF PageNumberPagination); CSV / PDF / Excel exports are never paged.

> The front end can keep all its screens. Only its data layer changes; the evaluation and heating model move to the server so every user, report and export sees the same numbers.

10. Security and operations

- **Passwords**: hashed by Django; users change their own on My profile (current password required); reset by an administrator or by e-mail link. The demo accounts in the front end must not be carried over.
- **Sessions**: HttpOnly, Secure, SameSite=Lax cookie; CSRF on all unsafe requests; sign-out and disabling a user end their sessions.
- **Brute force**: rate-limit sign-in (e.g. django-axes, 5 failures per 15 min) and log failures to the activity log.
- **Devices**: one API key per device, stored hashed, sent over HTTPS only; revoke from the admin.
- **Audit**: ActivityEntry is append only; restrict the Django admin to read-only for it.
- **Time**: store UTC; the UI shows local time (Asia/Kolkata).
- **Retention**: raw readings (one per real-time interval) 30 days; minute trend, tests and activity kept (archive yearly).
- **Backups**: nightly PostgreSQL dump plus point-in-time recovery; test a restore monthly.
- **Deployment**: Docker images for web (ASGI), Celery worker, Celery beat; Nginx serving the static pages and proxying /api and /ws.
- **Monitoring**: alert when a device stops sending while a test is running, and on ingestion errors.

10.1 Runtime services (what runs on the server)

Service	Software	Does	Notes
web	Django 5 on an ASGI server (Uvicorn or Daphne)	REST API /api/v1, WebSockets /ws, device ingestion, Django admin	2+ processes behind Nginx; stateless, so it scales out
worker	Celery worker	Evaluation, background jobs (5.3), exports, alerts	Same code image as web
beat	Celery beat	Starts the scheduled jobs of section 5.3	Exactly one instance
redis	Redis 7	Channels layer (live push), Celery broker, evaluation cache	Persistence on; private network only
db	PostgreSQL 16	All data (section 4)	Nightly dump + point-in-time recovery; private network only
nginx	Nginx	HTTPS (TLS certificate), serves the front-end pages, proxies /api and /ws to web	HTTP redirected to HTTPS; WebSocket upgrade headers

10.2 Settings each service needs (environment)

Variable	Example	Used for
DJANGO_SECRET_KEY	(long random)	Signing sessions and CSRF; never in the code
DATABASE_URL	postgres://tt:…@db:5432/tractiontherm	PostgreSQL connection
REDIS_URL	redis://redis:6379/0	Channels layer, Celery broker, cache
ALLOWED_HOSTS / CSRF_TRUSTED_ORIGINS	tt.railshed.in / https://tt.railshed.in	The site's host name
TIME_ZONE	Asia/Kolkata	Times shown on pages and reports (stored as UTC)
EMAIL_HOST / EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	smtp.railshed.in	Password-reset e-mails and alerts
ALERT_EMAILS	shed-supervisor@railshed.in	Who gets device-offline and ingestion alerts
DEBUG	False	Always False in production

11. Build plan

Phase	Scope	Done when
1. Foundation	Project, apps, custom user, roles and permissions, sign-in, /auth/me/, change password, admin	Users sign in and change their password on the server; the UI hides buttons from /auth/me/
2. Registry	Devices, motors, settings endpoints and validation	Devices, Motors and Settings pages read and write the API
3. Tests	TestRun, Test ID sequence, switch on, start, stop, assign, claim, channel, terminals, Edit test	All test actions work from the UI with the same rules
4. Telemetry	Ingest API, readings, minute trend, offline detection, WebSockets	A real device streams and Monitor / Motor pages update live
5. Evaluation	Checks, settle rule, heating model, results per test	Server conditions match the current UI on recorded data
6. Results and audit	Results queries with calculated results, report sheets as PDF and Excel, CSV export, activity with device tags	Results and Activity pages run on the API

Phase	Scope	Done when
7. Hardening	Rate limits, backups, retention, deployment, load test (10 devices × 2 channels)	Ready for the shed

Acceptance tests to port

The front end already has browser tests for every flow (sign-in, switch on, add details, assign, stop / restart, change channel and terminals, Edit test, Test ID rules, results per test, per-device activity, permissions for all four roles, page connectivity). Re-run them against the API build; the expected behaviour should not change.